MODULE

09

Data structures deep dive

Pick the right container and your code is fast. Pick the wrong one and it crawls.

YOU WILL LEARN

Big-O in plain words	01_big_o.py
Lists vs tuples	02_lists_and_tuples.py
Dicts & sets (hashing)	03_dicts_and_sets.py
collections	04_collections.py
dataclasses	05_dataclasses.py
Enums	07_enums.py
heapq & bisect	08_heapq_and_bisect.py
Interview Q&A	INTERVIEW.md

WHY BIG-O

Why the container matters

THE PROBLEM

Your login check does **if email in banned_emails**. With 10 users it's instant. With 1,000,000 users and a **list**, every login waits.

Same code, same result, but one choice is **50,000× slower**.

THE IDEA

Big-O describes how the work grows when the data grows. Learn 4 of them and you can choose well.

STEP BY STEP BIG-O

The 4 you need to know

BIG-O	PLAIN WORDS	EXAMPLE	1,000,000 ITEMS
O(1)	same speed, any size	`d[key]`, `x in set`	1 step
O(log n)	halve the problem each step	binary search, `bisect`	~20 steps
O(n)	look at every item once	`x in list`, `sum()`	1,000,000 steps
O(n²)	every item × every item	nested loops over the same data	10^{12} steps ⚠

> Interview habit: after every solution, say its **time** and **space** complexity.

TRY IT BIG-O

Measure it: list vs set lookup

```python
import timeit

emails_list = [f"user{i}@mail.com" for i in range(100_000)]
emails_set = set(emails_list)
target = "user99999@mail.com"           # the worst case for a list

t_list = timeit.timeit(lambda: target in emails_list, number=200)
t_set = timeit.timeit(lambda: target in emails_set, number=200)
print(f"list: {t_list * 1000:7.1f} ms")
print(f"set : {t_set * 1000:7.3f} ms")
print(f"set is ~{t_list / t_set:,.0f}x faster")
```

OUTPUT

```
list:   182.4 ms
set :   0.006 ms
set is ~30,000x faster
```

1. A list checks items **one by one**: O(n).
2. A set jumps straight to the answer: O(1).
3. Numbers vary per machine; the **gap** doesn't.

STEP BY STEP LISTS VS TUPLES

Same look, different jobs

	LIST [ ]	TUPLE ( )
Can change?	yes: append, remove, sort	**no** (immutable)
Typical use	many items of the **same kind**	one record with **fixed fields**
Example	`["lamp", "cable", "plug"]`	`("Vishal", "Augsburg", 31)`
Dict key / in a set?	no (not hashable)	yes, if its items are hashable
Fast operations	index, append, pop at the end: O(1)	index: O(1); a bit smaller and faster
Slow operations	insert / pop at the **front**: O(n)	—

> Need a queue (add at the back, take from the front)? Use **collections.deque**, not a list.

PICTURE IT DICTS AND SETS

How a dict finds a key instantly

🔑 Like a **wall of numbered lockers**. The key's **hash** says which locker to open, so Python never searches. That's why keys must be **hashable** (they must not change).



Sets use the same lockers, but store only keys: perfect for "have I seen this?".

TRY IT DICTS AND SETS

Everyday dict and set moves

```
stock = {"lamp": 3, "cable": 0}
print(stock.get("plug", 0))              # no KeyError
stock.setdefault("plug", 5)
print(stock)

a = {"python", "sql", "docker"}
b = {"python", "react"}
print(a & b, a | b, a - b, sep="  |  ")

try:
    {[1, 2]: "list as key"}
except TypeError as e:
    print("TypeError:", e)
```

```
OUTPUT
0
{'lamp': 3, 'cable': 0, 'plug': 5}
{'python'}  |  {'python', 'docker', 'react',
'sql'}  |  {'docker', 'sql'}
TypeError: unhashable type: 'list'
```

1. **.get(key, default)** instead of try/except KeyError.
2. Set maths: **&** both, **|** either, **-** difference.
3. Mutable things (lists, dicts) can't be keys.

COMMON MISTAKE COPIES

Shallow copy surprise

✗ WRONG

```
team = {"members": ["vishal", "li"]}
backup = team.copy()    # shallow!
backup["members"].append("sam")
print(team["members"])
# ['vishal', 'li', 'sam'] ???
```

.copy() copies the outer dict, but the inner list is **shared**. Changing it through the copy changes the original.

✓ RIGHT

```
import copy
backup = copy.deepcopy(team)
backup["members"].append("sam")
print(team["members"])
# ['vishal', 'li']
```

Use **copy.deepcopy** when the data is nested and you will change the copy. A favourite interview question.

TRY IT COLLECTIONS

Specialised containers save code

```
from collections import Counter, defaultdict, deque, namedtuple

words = "to be or not to be".split()
print(Counter(words).most_common(2))

by_letter = defaultdict(list)
for w in words:
    by_letter[w[0]].append(w)          # no "if key not in"
print(dict(by_letter))

recent = deque(maxlen=3)               # keeps the last 3
for page in ["/home", "/shop", "/cart", "/pay"]:
    recent.append(page)
print(list(recent))

Point = namedtuple("Point", "x y")
print(Point(3, 4).x)
```

OUTPUT

```
[('to', 2), ('be', 2)]
{'t': ['to', 'to'], 'b': ['be',
'be'], 'o': ['or'], 'n': ['not']}
['/shop', '/cart', '/pay']
3
```

1. **Counter**: count anything in one line.
2. **defaultdict**: missing keys get a default.
3. **deque**: O(1) at both ends; **maxlen** drops old items.

TRY IT DATACLASSES

Classes for data without boilerplate

```python
from dataclasses import dataclass, field

@dataclass(frozen=True)
class Money:
    amount: float
    currency: str = "EUR"

@dataclass
class Order:
    id: int
    items: list[str] = field(default_factory=list)   # not = []!
    total: Money = Money(0)

o = Order(1, ["lamp"], Money(49.0))
print(o)
print(o == Order(1, ["lamp"], Money(49.0)))
```

OUTPUT

```
Order(id=1, items=['lamp'],
total=Money(amount=49.0,
currency='EUR'))
True
```

1. You get **__init__**, **__repr__** and **__eq__** for free.
2. **frozen=True** = immutable and hashable.
3. Mutable defaults need **field(default_factory=list)**.

TRY IT ENUMS

A fixed set of named options

```python
from enum import StrEnum

class Status(StrEnum):
    PENDING = "pending"
    PAID = "paid"
    SHIPPED = "shipped"

s = Status("paid")              # from the database / JSON
print(s, s == "paid", s is Status.PAID)
print([x.value for x in Status])

try:
    Status("payed")             # typo: fails loudly
except ValueError as e:
    print("ValueError:", e)
```

OUTPUT

```
paid True True
['pending', 'paid', 'shipped']
ValueError: 'payed' is not a valid Status
```

1. Typos in plain strings fail **silently**; enum typos fail **loudly**.
2. **StrEnum** (3.11+) compares equal to its string: JSON and DB friendly.

TRY IT HEAPQ AND BISECT

Always get the most urgent item first

```
import bisect
import heapq

jobs = []
heapq.heappush(jobs, (3, "send newsletter"))
heapq.heappush(jobs, (1, "charge card"))
heapq.heappush(jobs, (2, "email receipt"))
print(heapq.heappop(jobs))              # smallest priority first

print(heapq.nlargest(2, [40, 5, 99, 12, 70]))

scores = [10, 20, 30, 40]
bisect.insort(scores, 25)               # stays sorted
print(scores, bisect.bisect_left(scores, 30))
```

OUTPUT

```
(1, 'charge card')
[99, 70]
[10, 20, 25, 30, 40] 3
```

1. A **heap** pops the smallest in O(log n): task queues, top-k, Dijkstra.
2. **bisect** searches and inserts in sorted lists in O(log n).

IN REAL JOBS DATA STRUCTURES

Where you'll use them at work

Dedupe and look up fast

```
seen = set()
if order_id in seen: return   # idempotency
```

Count events for a dashboard

```
Counter(e.status for e in events)
```

Group rows for a report

```
by_user = defaultdict(list)
for o in orders: by_user[o.user_id].append(o)
```

Typed DTOs between layers

```
@dataclass(frozen=True)
class OrderDTO: id: int; total: Decimal
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How does a Python dict work, and why are lookups O(1)?”

- A dict is a **hash table**: Python calls **hash(key)** and uses it to pick a slot directly.
- So lookup, insert and delete are **O(1) on average**; collisions are rare and handled.
- Keys must be **hashable**: immutable, with a stable hash (str, int, tuple; not list or dict).
- Since Python 3.7 dicts **keep insertion order**.
- Sets are the same idea without values.

```
d = {"a": 1}
print(hash("a"))      # used to find the slot
d[(1, 2)] = "ok"      # tuple: hashable
d[[1, 2]] = "no"      # TypeError: unhashable
```

Follow-up they ask next: ”What happens if two keys have the same hash?” (a collision: Python probes for another slot and compares with == to find the right key).

INTERVIEW SAY THIS IN THE INTERVIEW

“List vs tuple? Shallow vs deep copy?”

- **List**: mutable, for many items of the same kind.
  Tuple: immutable, a fixed record, can be a dict key.
- Tuples are a bit smaller and faster, and signal ”this won’t change”.
- A **shallow copy** (**list(x)**, **.copy()**, **x[:]**) copies the outer object but **shares** nested objects.
- A **deep copy** (**copy.deepcopy**) copies everything recursively.

```
a = [[1, 2], [3]]
b = a.copy()
b[0].append(99)
print(a)        # [[1, 2, 99], [3]]  shared!
```

> **Follow-up they ask next:** ”Which data structure would you use for an LRU cache?” (an OrderedDict or dict + deque; or just functools.lru_cache).

RECAP REMEMBER THESE

Module 9 in 6 lines

1. Know **O(1)**, **O(log n)**, **O(n)**, **O(n²)**, and say them out loud.
2. Membership checks: **set**, not list. Lookups by key: **dict**.
3. List = many similar items; tuple = a fixed record; queue = **deque**.
4. **.copy()** is shallow; nested data needs **deepcopy**.
5. **Counter**, **defaultdict**, **dataclass**, **StrEnum** remove boilerplate.
6. **heapq** for "most urgent first", **bisect** for sorted lists.

PRACTICE NOW

Do exercises/ex09_library_tracker.py: pick the right structure for each job.